

Write Quarto books with Bioconductor

Table of contents

What are BiocBooks?	4
What is this {BiocBook} package?	6
Main features of BiocBooks	7
Fully compatible with the <i>Bioconductor Build System</i>	7
Automated versioning of Docker images	7
Automated versioning of the online book	8
RStudio Server	8
Acknowledgments	9
Session info	10
References	13
Preamble	14
1 BiocBook versioning system	15
1.1 Continuous Integration and Continuous Delivery	15
1.1.1 From local to Github	15
1.1.2 Package submission to Bioconductor	16
1.1.3 New Bioconductor releases	18
1.1.4 Updates	19
1.2 Access to versioned Docker and online book	20
1.2.1 Docker images versioning	20
1.2.2 Website versioning	21
1.3 Does this really work?	21
1.4 So what packages can I use?	21
1.5 Session info	22
2 Writing a {BiocBook} package	24
2.1 Register a Github account in R	24
2.1.1 Creating a new Github token	24
2.1.2 Register your new token in R	24
2.1.3 Double check you are logged in	25

2.2	BiocBook workflow	25
2.2.1	Initiate a {BiocBook} package	25
2.2.2	Edit new BiocBook chapters	30
2.2.3	Edit assets	30
2.2.4	Previewing and publishing changes	31
2.3	Writing features	31
2.3.1	Executing code	31
2.3.2	Creating data object	32
2.3.3	Adding references	33
2.4	Session info	33
3	Containerizing and Publishing against specific Bioconductor release versions	35
3.1	For a {BiocBook} package not currently in Bioconductor	35
3.2	For a {BiocBook} package accepted in Bioconductor > 6 months ago	35
3.3	Session info	36
4	Executing python code	37
4.1	Declaring and installing the python environment	37
4.2	Where conda itself comes from	38
4.3	A worked example	38
4.4	Bioconda packages	40
4.5	Session info	40
5	{BiocBook}-based books vs. {rebook}-based books	43
5.1	Differences with {rebook}-based books	43
5.2	BiocBook features missing from rebook	44
5.3	rebook features missing from BiocBook	44
5.4	Session info	45

What are BiocBooks?

Package: BiocBookDemo **Authors:** Jacques Serizay [aut, cre] **Compiled:** 2026-09-04
Package version: 1.11.1 **R version:** R version 4.6.1 (2026-06-24) **BioC version:** 3.24
License: MIT + file LICENSE

BiocBooks are **package-based, versioned online books** with a **supporting Docker image** for each book version.

A BiocBook can be created by authors (e.g. R developers, but also scientists, teachers, communicators, ...) who wish to:

1. *Write*: compile a **body of biological and/or bioinformatics knowledge**;
2. *Containerize*: provide **Docker images** to reproduce the examples illustrated in the compendium;
3. *Publish*: deploy an **online book** to disseminate the compendium;
4. *Version*: **automatically** generate specific online book versions and Docker images for specific [Bioconductor releases](#).

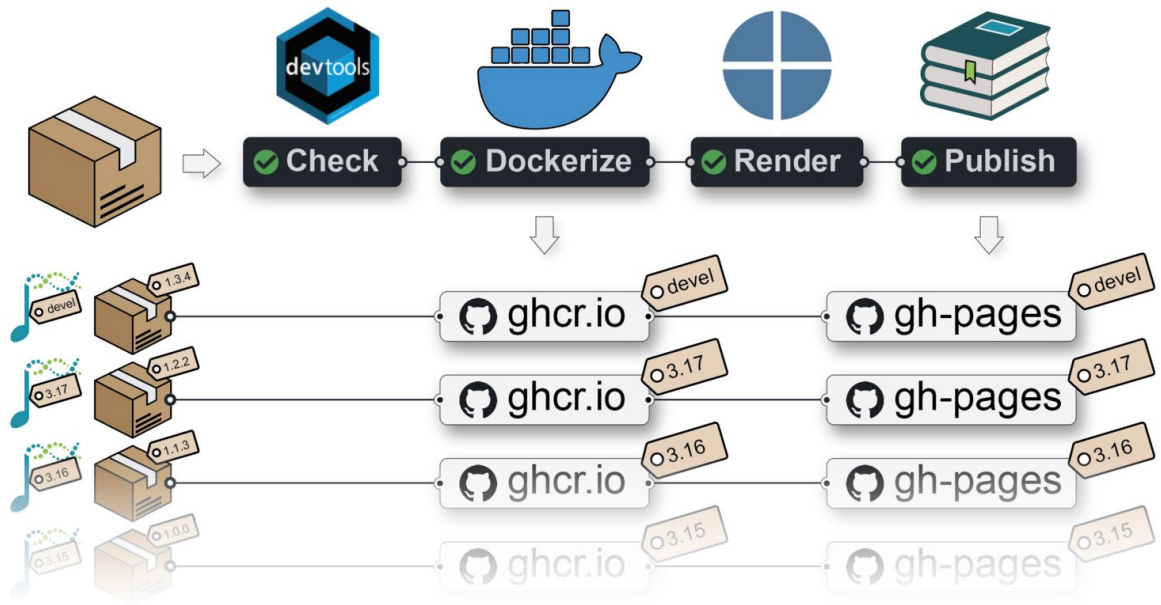
Tip

A {BiocBook}-based package hosted on **GitHub** with a branch named `RELEASE_X_Y` provides:

- A **Docker image**: hosted on ghcr.io;
- An **online book** (a.k.a website): hosted on the GitHub repository `gh-pages` branch;

Both are built against the specific Bioconductor release `X.Y`.

A {BiocBook}-based package submitted to **Bioconductor** also lead to the online book being independently built by the **Bioconductor Build System (BBS)** and deployed to https://bioconductor.org/books/<bioc_version>/<pkg>/



What is this {BiocBook} package?

The {BiocBook} **package** offers a streamlined approach to creating BiocBooks, with several important benefits:

- The author creates a {BiocBook}-based package without leaving R;
- The author writes book chapters in `pages/*.qmd` files using enhanced markdown;
- The author can submit its {BiocBook}-based package to Bioconductor.

The containerization and publishing of the new {BiocBook}-based package is automated:

- A Github Action workflow **generates different Docker images** for different Bioconductor releases, with the packages used in the book pre-installed;
- A Github Action workflow **publishes different book versions** for different Bioconductor releases.

Main features of BiocBooks

Fully compatible with the *Bioconductor Build System*

When a {BiocBook}-based package is accepted into Bioconductor, it is automatically integrated into the *Bioconductor Build System* (BBS).

This means that it is getting built using R CMD build `--keep-empty-dirs --no-resave-data ..`. This triggers the rendering of the book contained in `/inst/`. Book packages built by the BBS are then automatically deployed and are eventually available at https://bioconductor.org/books/<bioc_version>/<pkg>/.

Automated versioning of Docker images

A separate Docker image is built for each branch (named `devel` or `RELEASE_X_Y`) of a {BiocBook}-based **Github repository**.

Each Docker image provides pre-installed R packages:

- Bioconductor release `X.Y`;
- Specific book dependencies from Bioconductor release `X.Y` (listed in `DESCRIPTION`);
- The book package itself

The Docker images also include a `python` environment, named `BiocBook`, in which all the packages listed in `requirements.yml` are installed, and which `reticulate` picks up automatically in any R session started from the image (see Chapter 4).

For example, Docker images built from the {BiocBookDemo} package repository are available here:

ghcr.io/js2264/biocbookdemo

 Get started now

You can get access to all the packages used in this book in < 1 minute, using this command in a terminal:

Listing 0.1 bash

```
docker run -it ghcr.io/js2264/biocbookdemo:devel R
```

Automated versioning of the online book

Regardless of whether the book package is submitted to Bioconductor, a Github Actions workflow publishes individual online books for each branch (named `devel` or `RELEASE_X_Y`) of a BiocBook-based **Github repository**.

For example, the online book version matching the `devel` version of the {BiocBook} package is available from:

<http://js2264.github.io/BiocBookDemo/devel/>

RStudio Server

An RStudio Server instance based on a specific Bioconductor `<version>` (`devel` or `RELEASE_X_Y`) can be initiated from the corresponding Docker image as follows:

Listing 0.2 bash

```
docker run \  
  --volume <local_folder>:<destination_folder> \  
  -e PASSWORD=OHCA \  
  -p 8787:8787 \  
  ghcr.io/<github_user>/<biocbook_repo>:<version>
```

The initiated RStudio Server instance will be available at <https://localhost:8787>.

Further instructions regarding Bioconductor-based Docker images are available [here](#).

Acknowledgments

This work was inspired by and closely follows the strategy used in coordination by the Bioconductor core team and Aaron Lun to submit book-containing packages (from the `OSCA` series as well as `SingleR` and `csaw` books).

- @OSCA
- @SingleR
- @csaw

This package was also inspired by the `*down` package series, including:

- @knitr
- @bookdown
- @pkgdown

Session info

 Click to expand

References

Preamble

This page is kept empty on purpose.

1 BiocBook versioning system

⚠ Important points

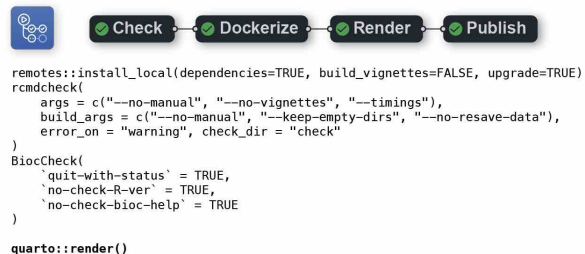
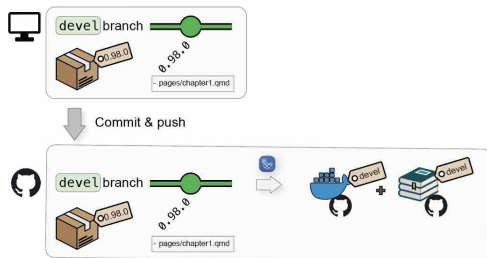
- Any **package** built using the {BiocBook} package is itself a {BiocBook}-based package;
- As such, it follows the release schedule from Bioconductor;
- The `pages/` folder contains a number of pages which are rendered as a **website** using Quarto;
- The **name** of the branch (`devel` or `RELEASE_X_Y`) is *crucial*, as it is used to select a Bioconductor version to 1) build a **Docker image** for the {BiocBook}-based package and 2) render the BiocBook **website**.

1.1 Continuous Integration and Continuous Delivery

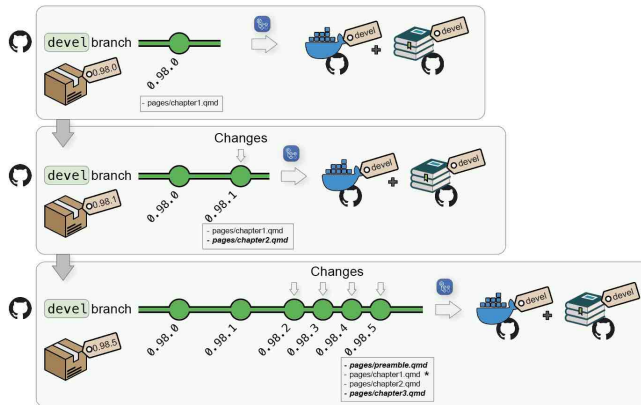
1.1.1 From local to Github

When pushing a {BiocBook}-based **package** from a local `devel` branch to Github, (e.g. when writing new articles), two jobs are automatically triggered from a single Github Actions workflow:

1. First, a **Docker image** will be created to build the {BiocBook} package against the Bioconductor `devel` branch and push the resulting image to `ghcr.io/<github_user>/<package_name>:devel`
2. Then, this image will be used to render the BiocBook **website** and deploy it to `https://<github_user>.github.io/<package_name>/devel/`



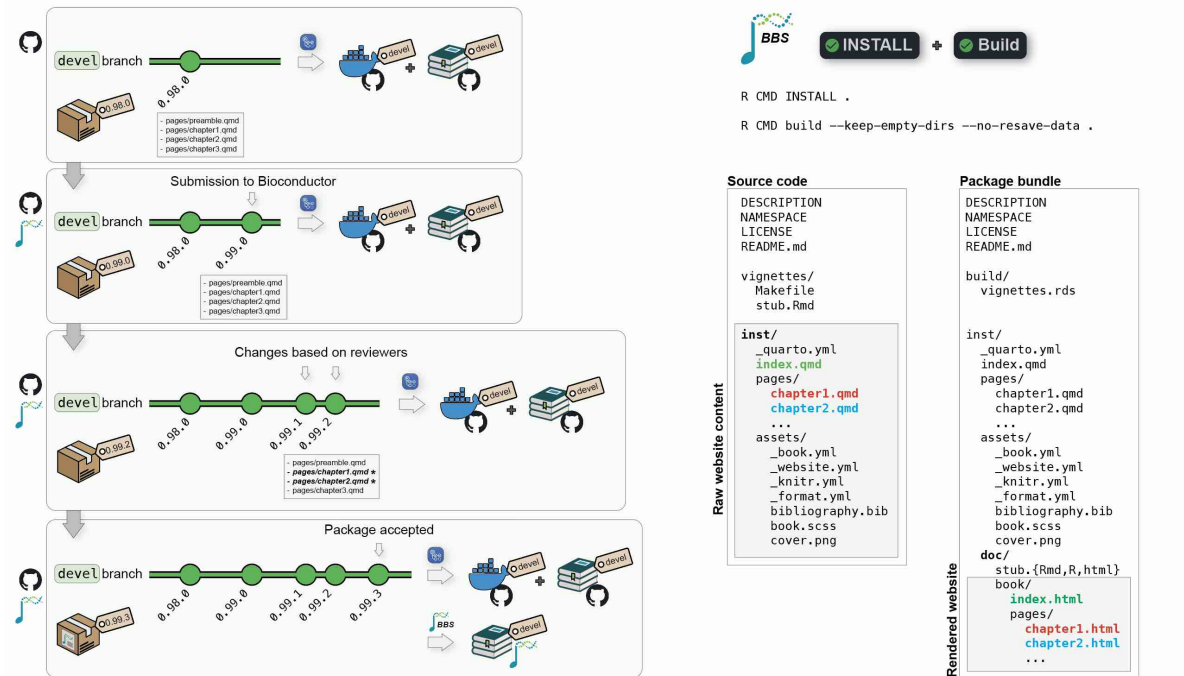
Additional commits on the `devel` branch will trigger regeneration of the `devel` Docker image and the online book `devel` version.



1.1.2 Package submission to Bioconductor

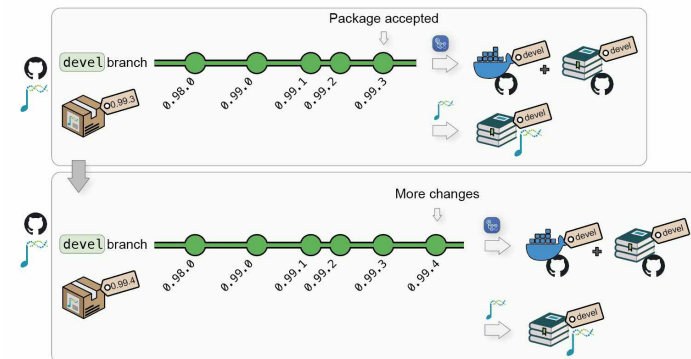
Submission to Bioconductor can follow the [same reviewing process](#) as other standard packages.

1. The author submits a book package to Bioconductor/Contributions;
2. Once review starts, the package gets tested by Bioconductor's *Single Package Builder* (*SPB*);
3. The author pushes changes to its Github repository; this will update the `devel` Docker image and the online book `devel` version;
4. Regularly, the author bumps versions and push to Bioconductor's `upstream` branch to trigger a new *SPB* test run;
5. Once the *SPB* returns no error/warnings, the package may be accepted;
6. When a book package is accepted, it starts being regularly built by the *Bioconductor Build System* (*BBS*).
7. The *BBS* build automatically renders the book and deploys it online.



At all time, the author's local `devel` branch should be synchronized with its Github remote `origin` as well as the Bioconductor `upstream` remote.

- Any commit pushed to the remote Github `origin` remote will trigger a new Github Actions workflow and regenerate the `devel` Docker image and the online book served by Github.
- Any commit pushed to the remote Bioconductor `upstream` remote will result in a new build by the **BBS** and an update of the **Bioconductor's** hosted book.

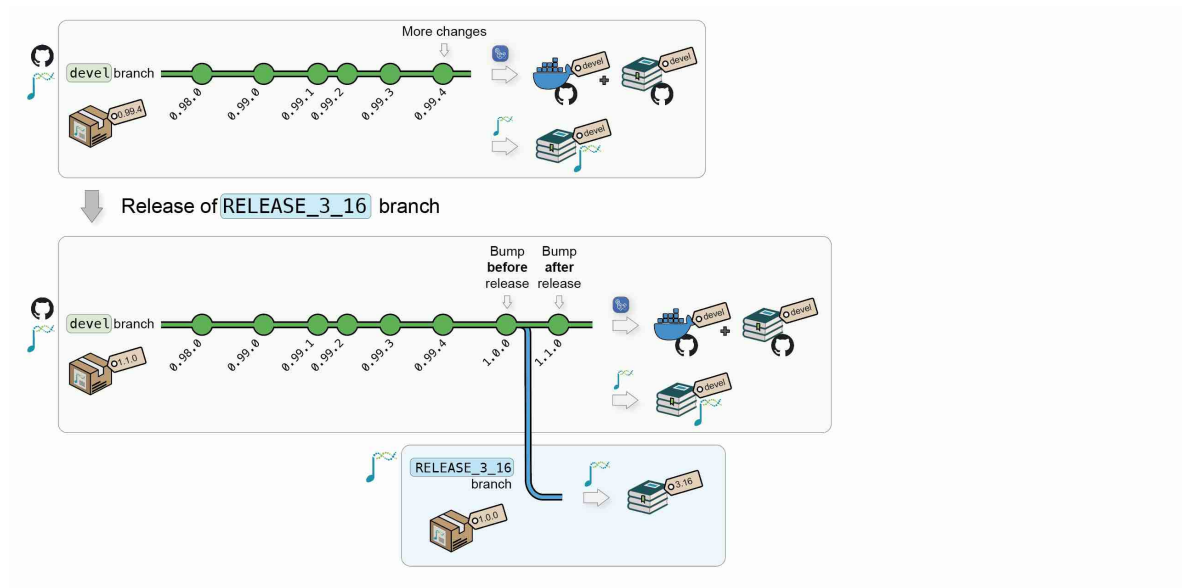


1.1.3 New Bioconductor releases

When Bioconductor releases a version X.Y, the core team will automatically create a new upstream: <package_name>@RELEASE_X.Y. This will automatically trigger new builds of the **Bioconductor's hosted book** against the new release.

When this occurs, the upstream branch can also be pulled to origin: <package_name>@RELEASE_X.Y to:

1. Create a **Docker image** @ ghcr.io/<github_user>/<package_name>:RELEASE_X.Y, with the book **package** installed using Bioconductor X.Y;
2. Publish the book **website** to https://<github_user>.github.io/<package_name>/X.Y/, using packages from Bioconductor X.Y.



i Note

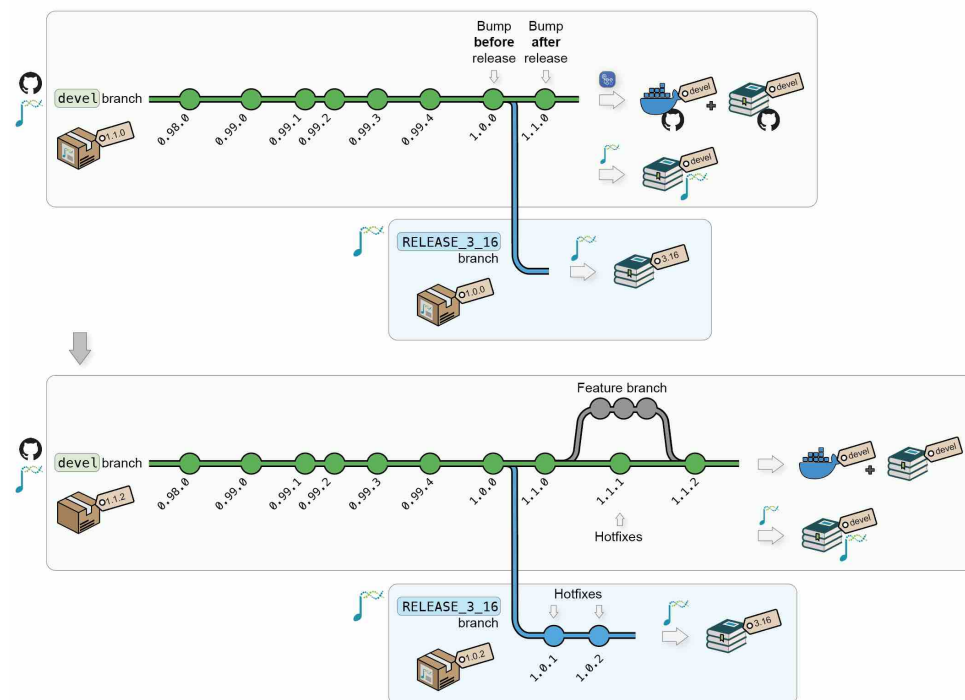
A {BiocBook}-based **package** can follow its own release life cycle if the author does not intend to submit it to Bioconductor.

If the author of a {BiocBook}-based **package** intends to submit this package/book/website to Bioconductor, the [Bioconductor submission and release life cycle](#):

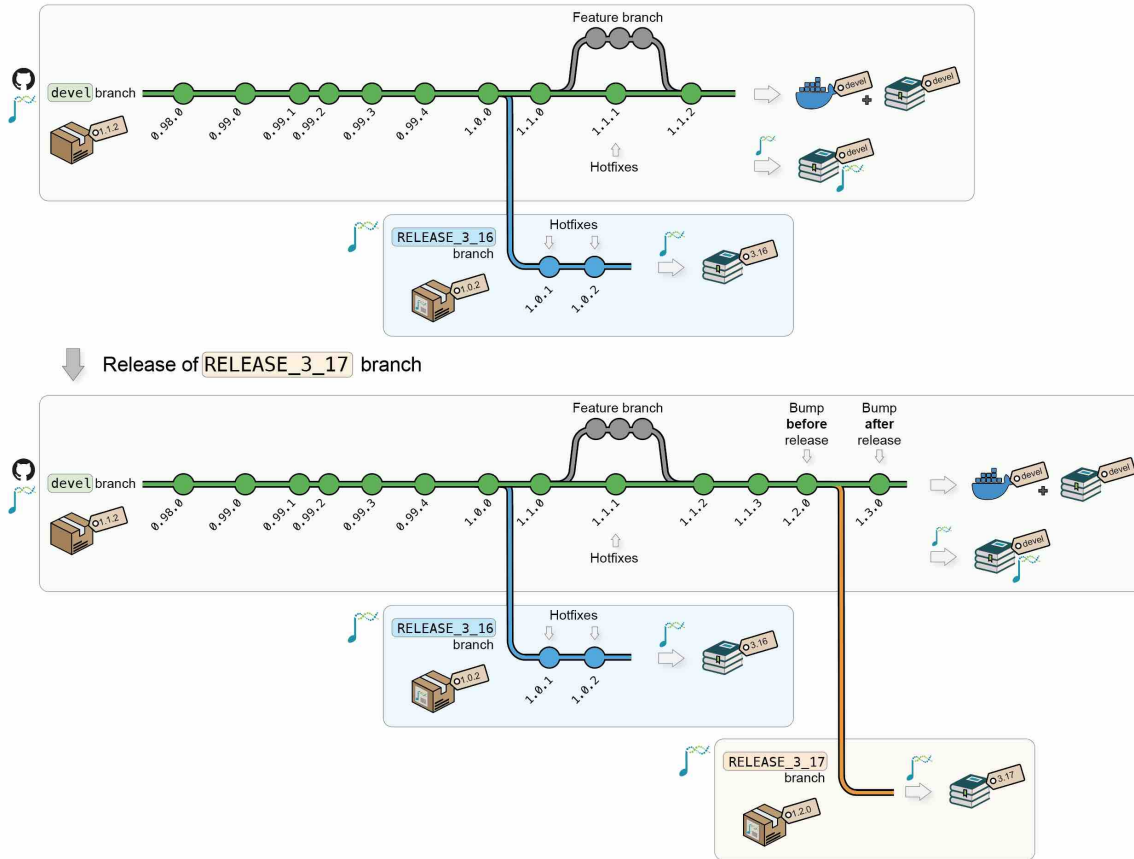
- When developing a {BiocBook}-based **package** (at the submission and during review), the **package** version should be between 0.99.0 and 1.0.0.
- Once the submission is accepted and Bioconductor releases a new version X.Y, the {BiocBook}-based **package** version will automatically be bumped to 1.0.0 in Bioconductor release X.Y and to 1.1.0 in the continuing Bioconductor **devel**.

1.1.4 Updates

After new releases, updates and/or hot fixes can still be made, both on the latest Bioconductor release and on the `devel` branch. New commits will automatically trigger the regeneration of the Docker image and the online book for the modified branch(es).



Additional Bioconductor releases will generate new versions of the Docker image and of the online book.



1.2 Access to versioned Docker and online book

1.2.1 Docker images versioning

The different versions of a BiocBook **Docker image** are available at the following URL:

`ghcr.io/<github_user>/<package_name>`

For example, for this package (`{BiocBookDemo}`), the following **Docker image** versions are available:

- ghcr.io/js2264/biocbookdemo:devel
- ghcr.io/js2264/biocbookdemo:3.17
- ghcr.io/js2264/biocbookdemo:3.16
- ghcr.io/js2264/biocbookdemo:3.15

1.2.2 Website versioning

The different versions of a BiocBook **website** are hosted at the following URL:

`https://<github_user>.github.io/<package_name>/<version>`

For example, for this package (`{BiocBookDemo}`), the following **website** versions are available:

- <https://js2264.github.io/BiocBookDemo/devel/>
- <https://js2264.github.io/BiocBookDemo/3.17/>
- <https://js2264.github.io/BiocBookDemo/3.16/>
- <https://js2264.github.io/BiocBookDemo/3.15/>

1.3 Does this really work?

The `BIOCONDUCTOR_DOCKER_VERSION` variable is set in all Bioconductor Docker images. For instance, this version of the `BiocBookDemo` package online book relies on:

```
Sys.getenv("BIOCONDUCTOR_DOCKER_VERSION")  
## [1] "3.24.14"
```

Tip

Note that this variable will always match the `X.Y` version returned by `BiocVersion` used to render the online book.

```
packageVersion("BiocVersion")  
## [1] '3.24.0'
```

1.4 So what packages can I use?

Any package that has been released in the Bioconductor version you are using (in this book version, this is 3.24.0).

The `BiocBaseUtils` package is available in Bioconductor since 3.16, while the `BiocHail` package was only made available in 3.17. `CuratedAtlasQueryR` has recently been accepted in 3.18 (current `devel`, on Fri Sep 4 21:36:51 2026). Let's check this!

```

packageVersion("BiocVersion")
## [1] '3.24.0'
BiocManager::available("BiocBaseUtils")
## [1] "BiocBaseUtils"
BiocManager::available("BiocHail")
## [1] "BiocHail"
BiocManager::available("CuratedAtlasQueryR")
## character(0)

```

1.5 Session info

```

sessioninfo::session_info()
## - Session info -----
## setting value
## version R version 4.6.1 (2026-06-24)
## os Ubuntu 24.04.4 LTS
## system x86_64, linux-gnu
## ui X11
## language (EN)
## collate C
## ctype en_US.UTF-8
## tz Etc/UTC
## date 2026-09-04
## pandoc 3.10.2 @ /usr/bin/ (via rmarkdown)
## quarto 1.9.38 @ /usr/local/bin/quarto
##
## - Packages -----
## package * version date (UTC) lib source
## BiocManager 1.30.27 2025-11-14 [2] CRAN (R 4.6.1)
## cli 3.6.6 2026-04-09 [2] RSPM (R 4.6.0)
## digest 0.6.39 2025-11-19 [2] RSPM (R 4.6.0)
## evaluate 1.0.5 2025-08-27 [2] RSPM (R 4.6.0)
## fastmap 1.2.0 2024-05-15 [2] RSPM (R 4.6.0)
## htmltools 0.5.9 2025-12-04 [2] RSPM (R 4.6.0)
## jsonlite 2.0.0 2025-03-27 [2] RSPM (R 4.6.0)
## knitr 1.51 2025-12-20 [2] RSPM (R 4.6.0)
## otel 0.2.0 2025-08-29 [2] RSPM (R 4.6.0)
## rlang 1.3.0 2026-07-05 [2] RSPM (R 4.6.0)
## rmarkdown 2.32 2026-09-01 [2] RSPM (R 4.6.0)

```

```
## sessioninfo 1.2.4 2026-06-04 [2] RSPM (R 4.6.0)
## xfun 0.60 2026-07-09 [2] RSPM (R 4.6.0)
##
## [1] /tmp/RtmpopvETa/Rinst8e262e157d
## [2] /usr/local/lib/R/site-library
## [3] /usr/local/lib/R/library
##
## -----
```

2 Writing a {BiocBook} package

2.1 Register a Github account in R

Tip

Skip this section if your Github account is already registered. You can check this by typing:

```
gh::gh_whoami()
```

2.1.1 Creating a new Github token

```
usethis::create_github_token(  
  description = "BiocBook",  
  scopes = c("repo", "user:email", "workflow")  
)
```

This command will open up a new web browser. On the displayed Github page:

- Select an Expiration date;
- Make sure that at least `repo`, `user > user:email` and `workflow` scopes are selected;
- Click on “Generate token” at the bottom of the page;
- Copy your Github token displayed in the Github web page

2.1.2 Register your new token in R

```
gitcreds::gitcreds_set()
```

Paste your new Github token here and press “Enter”.

💡 Saving your Github token for later use

On Linux, `gitcreds` is generally not able to permanently store the provided Github token. For this reason, you may want to also add your Github token to `~/.Renviron` to be able to reuse it. You can edit the `~/.Renviron` by typing `usethis::edit_r_environ()`, and define the `GITHUB_PAT` environment variable:

Listing 2.1 .Renviron

```
GITHUB_PAT="<YOUR-TOKEN>"
```

2.1.3 Double check you are logged in

```
gh::gh_whoami()
```

2.2 BiocBook workflow

2.2.1 Initiate a {BiocBook} package

2.2.1.1 R

Creating a BiocBook in R is straightforward with the {BiocBook} package.

```
if (!require("BiocManager", quietly = TRUE)) install.packages("BiocManager")
if (!require("BiocBook", quietly = TRUE)) BiocManager::install("BiocBook")
library(BiocBook)
biocbook <- init("myBook")
```

The steps performed under the hood by `init()` are detailed in the console. Briefly, the following steps are followed:

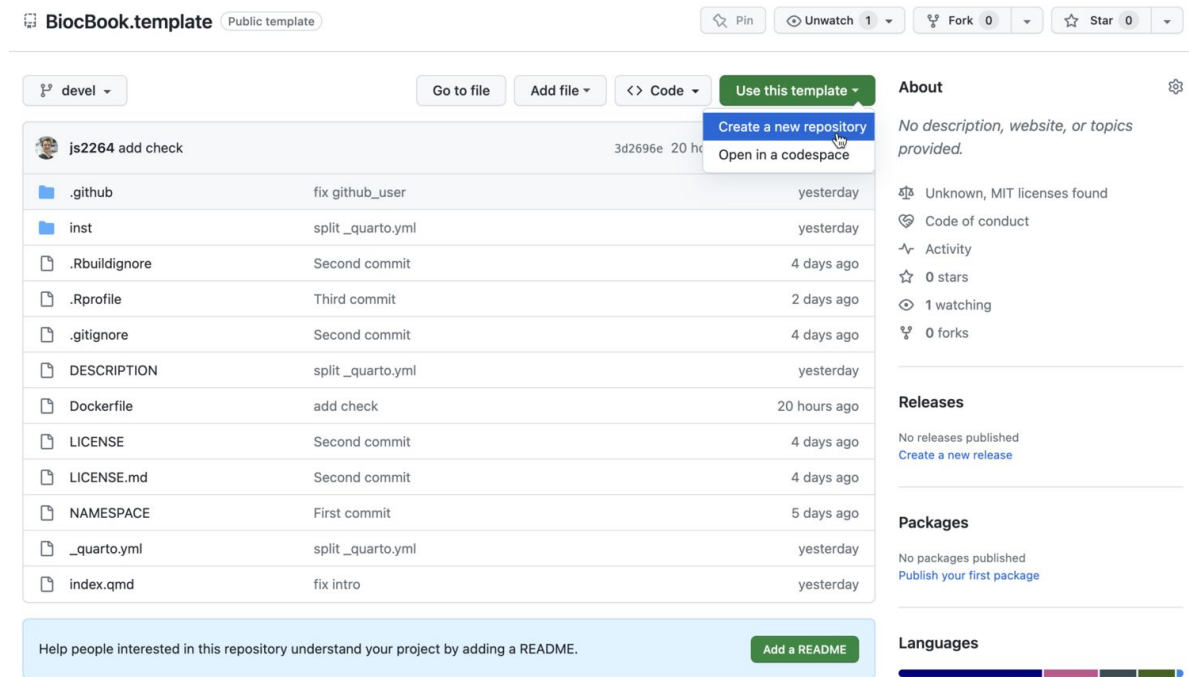
1. Creating a local git repository using the BiocBook package template
2. Fillout placeholders from the template
3. Push local commits to your Github account, creating a new GitHub repository

2.2.1.2 VS Code

⚠ This approach is significantly more hazardous. It is highly recommended to stick to the `init()` helper function from the `{BiocBook}` package.

Use the `{BiocBook.template}` package template

This template can be cloned from [js2264/BiocBook.template](https://github.com/js2264/BiocBook.template)



BiocBook.template Public template

Pin Unwatch 1 Fork 0 Star 0

devel

Go to file Add file <> Code Use this template

Create a new repository Open in a codespace

File	Commit	Time
js2264 add check	3d2696e	20 hours ago
.github	fix github_user	yesterday
inst	split_quarto.yml	yesterday
.Rbuildignore	Second commit	4 days ago
.Rprofile	Third commit	2 days ago
.gitignore	Second commit	4 days ago
DESCRIPTION	split_quarto.yml	yesterday
Dockerfile	add check	20 hours ago
LICENSE	Second commit	4 days ago
LICENSE.md	Second commit	4 days ago
NAMESPACE	First commit	5 days ago
_quarto.yml	split_quarto.yml	yesterday
index.qmd	fix intro	yesterday

Help people interested in this repository understand your project by adding a README. Add a README

About

No description, website, or topics provided.

Unknown, MIT licenses found

Code of conduct

Activity

0 stars

1 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Languages

Language	Percentage
Lua	51.8%
SCSS	19.4%
Dockerfile	13.2%
TeX	13.1%
R	2.5%

Create a new repo

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

Repository template

 js2264/BlocBook.template ▾

Start your repository with a template repository's contents.

☐ **Include all branches**

Copy all branches from js2264/BlocBook.template and not just the default branch.

Owner *

 js2264 ▾

Repository name *

/ MyBook

✔ MyBook is available.

Great repository names are short and memorable. Need inspiration? How about [studious-guide](#) ?

Description (optional)

☒  **Public**

Anyone on the internet can see this repository. You choose who can commit.

☐  **Private**

You choose who can see and commit to this repository.

 You are creating a public repository in your personal account.

Create repository

2.2.1.2.1 Enable Github Pages to be deployed

You will need to enable the Github Pages service for your newly created repository.

- Go to your new Github repository;
- Open the “Settings” tab;
- On the leftside bar, click on the “Pages” tab;
- Select the `gh-pages` branch and the `/docs` folder to deploy your Github Pages.

js2264 / MyBook

🔍

+

🕒

🔗

📧

👤

<> Code

🔖 Issues

🔗 Pull requests

🎬 Actions

📁 Projects

📖 Wiki

🛡 Security

📊 Insights

⚙ Settings

⚙ General

Access

👤 Collaborators

🔒 Moderation options

Code and automation

🌿 Branches

🏷 Tags

📄 Rules

🎬 Actions

🔗 Webhooks

🌐 Environments

💻 Codespaces

📁 Pages

GitHub Pages

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.

Build and deployment

Source

Deploy from a branch

Branch

Your GitHub Pages site is currently being built from the gh-pages branch. [Learn more about configuring the publishing source for your site.](#)

gh-pages /docs Save

Learn how to [add a Jekyll theme](#) to your site.

Custom domain

Enter VS Code editor by pressing .

EXPLORER

OPEN EDITORS

_book.yml inst/assets

MYBOOK [GITHUB]

.github

inst

assets

_book.yml

_format.yml

_knitr.yml

_website.yml

bibliography.bib

book.scss

cover.png

favicon.png

requirements.txt

extensions

_quarto.yml

.gitignore

.Rbuildignore

.Rprofile

DESCRIPTION

Dockerfile

index.qmd

LICENSE

LICENSE.md

NAMESPACE

_book.yml

inst > assets > _book.yml > {} book > {} sidebar

1 book:

2 title: "<Package_name>"

3 chapters:

4 index.qmd

5 cover-image: inst/assets/cover.png

6 favicon: inst/assets/favicon.png

7 sidebar:

8 tools:

9 icon: git

10 menu:

11 text: Source Code

12 url: https://github.com/<github_user>/<Package_name>/

13 text: Browse version `devel`

14 url: https://<github_user>.github.io/<Package_name>/devel/

15 style: "docked"

16 background: "light"

17 collapse-level: 5

18

28

Fillout placeholders

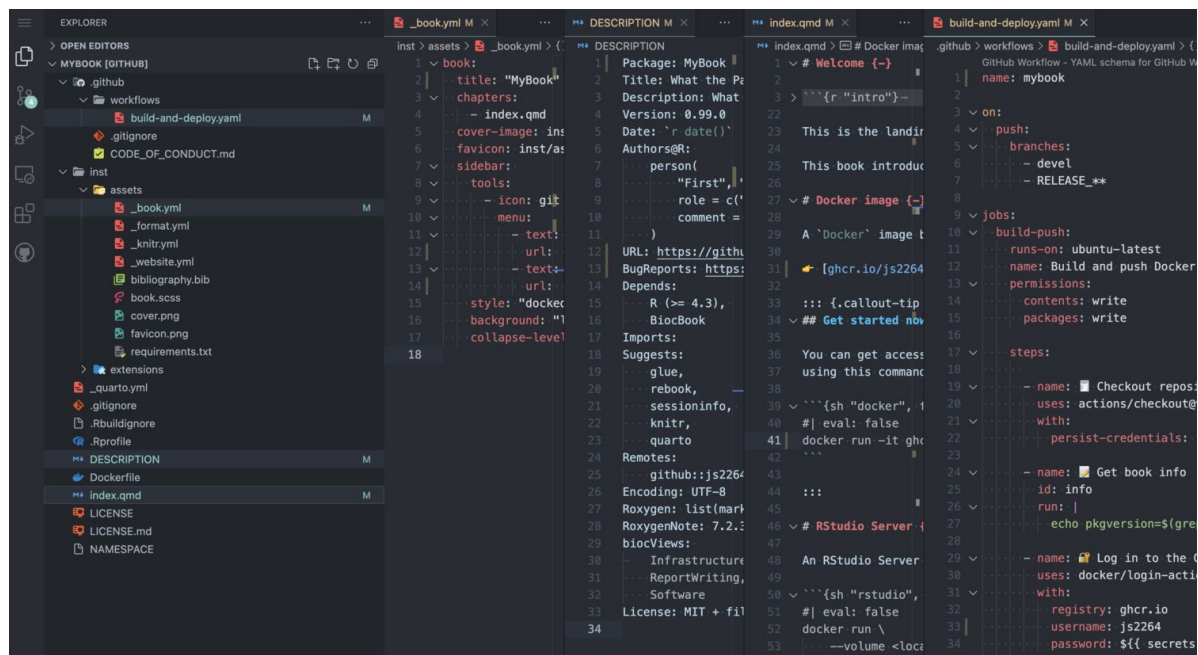
 Warning

Three types of placeholders need to be replaced:

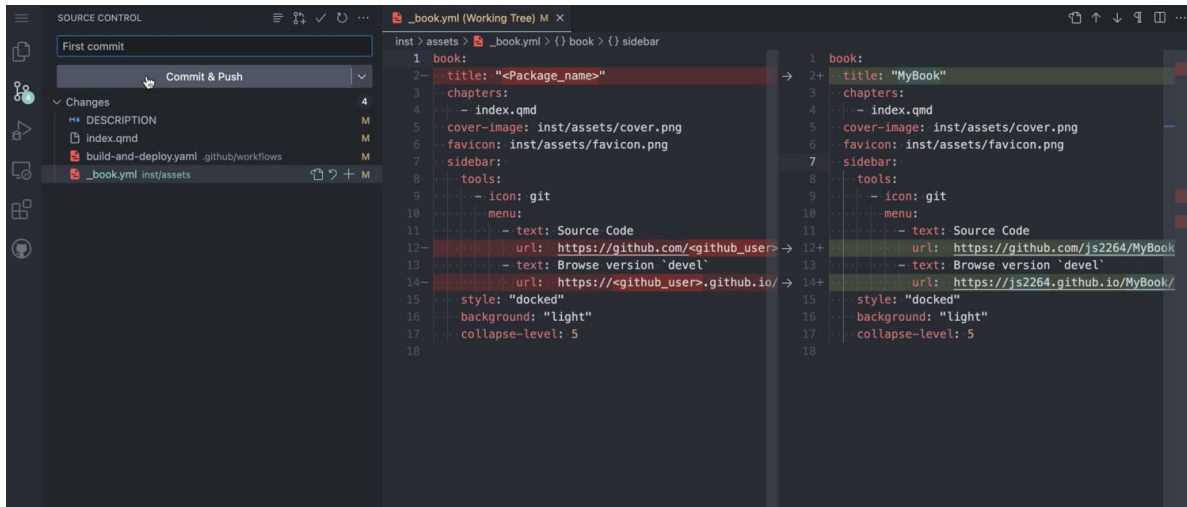
1. <Package_name>
2. <package_name>
3. <github_user>

Three different files contain these placeholders:

1. /inst/assets/_book.yml
2. /DESCRIPTION
3. /index.qmd



Commit changes



Clone the package to a local computer

2.2.2 Edit new BiocBook chapters

- `add_chapter(biocbook, title)` is used to write new chapters;
- `add_preamble(biocbook)` is used to add an unnumbered extra page after the Welcome page but before the chapters begin.

⚠ Don't forget to add any package used in the book pages to **Imports:** or **Suggests:** fields in DESCRIPTION.

This ensures that these packages are installed in the Docker image prior to rendering.

2.2.3 Edit assets

A BiocBook relies on several assets, located in `/inst/assets`:

- `_book.yml`, `_format.yml`, `_knitr.yml`, `_website.yml`
- `bibliography.bib`
- `book.scss`

To quickly edit these assets, use the corresponding `edit_*` functions:

```
edit_yaml(biocbook)
edit_bib(biocbook)
edit_css(biocbook)
```

2.2.4 Previewing and publishing changes

2.2.4.1 Previewing

While writing, you can monitor the rendering of your book live as follows:

```
preview(biocbook)
```

This will serve a local live rendering of your book.

2.2.4.2 Publishing

Once you are done writing pages of your new book, you should always commit your changes and push them to Github. This can be done as follows:

```
publish(biocbook, message = " Publish")
```

2.2.4.3 Check your published book and Dockerfiles

This connects to the Github repository associated with a local book and checks the existing branches and Dockerfiles.

```
status(biocbook)
```

2.3 Writing features

2.3.1 Executing code

It's super easy to execute actual code from any BiocBook page when rendering the BiocBook website.

2.3.1.1 R code

R code can be executed and rendered:

2.3.1.2 bash code

bash code can also be executed and rendered:

Listing 2.2 R

```
utils::packageVersion("BiocVersion")  
## [1] '3.24.0'
```

Listing 2.3 bash

```
find ../ -name "*.qmd"  
## ../index.qmd  
## ../pages/Chapter-4.qmd  
## ../pages/biocbook-vs-rebook.qmd  
## ../pages/Chapter-3.qmd  
## ../pages/Chapter-2.qmd  
## ../pages/preamble.qmd  
## ../pages/Chapter-1.qmd
```

2.3.1.3 python code

python code can also be executed and rendered:

Listing 2.4 python

```
import os  
os.getcwd()
```

python needs a little more care than R or bash, because the book is also re-rendered by the Bioconductor Build System, which provides no python packages. A book therefore installs its own: the packages are declared as a conda environment in `inst/requirements.yml` and installed at build time by `BiocBook::setup_python()`.

See [the chapter on executing python code](#) for the full story.

2.3.2 Creating data object

While writing chapters, you can save objects as `.rds` files to reuse them in subsequent chapters (e.g. [here](#)).

```
isthisworking <- "yes"  
saveRDS(isthisworking, 'isthisworking.rds')
```


2.3.3 Adding references

References can be listed as `.bib` entries in the bibliography file located in `inst/assets/bibliography.bib`. The references can be added in-line using the `@` notation, e.g. by typing `@serizay2023`, this will insert the following reference: `@serizay2023`.

2.4 Session info

```
sessioninfo::session_info()
## - Session info -----
##   setting      value
##   version      R version 4.6.1 (2026-06-24)
##   os           Ubuntu 24.04.4 LTS
##   system       x86_64, linux-gnu
##   ui           X11
##   language     (EN)
##   collate      C
##   ctype        en_US.UTF-8
##   tz           Etc/UTC
##   date         2026-09-04
##   pandoc       3.10.2 @ /usr/bin/ (via rmarkdown)
##   quarto       1.9.38 @ /usr/local/bin/quarto
##
## - Packages -----
##   package      * version date (UTC) lib source
##   cli           3.6.6   2026-04-09 [2] RSPM (R 4.6.0)
##   digest        0.6.39  2025-11-19 [2] RSPM (R 4.6.0)
##   evaluate      1.0.5   2025-08-27 [2] RSPM (R 4.6.0)
##   fastmap       1.2.0   2024-05-15 [2] RSPM (R 4.6.0)
##   htmltools     0.5.9   2025-12-04 [2] RSPM (R 4.6.0)
##   jsonlite      2.0.0   2025-03-27 [2] RSPM (R 4.6.0)
##   knitr         1.51    2025-12-20 [2] RSPM (R 4.6.0)
##   lattice       0.23-1  2026-08-12 [2] RSPM (R 4.6.0)
##   Matrix        1.7-6   2026-07-25 [2] RSPM (R 4.6.0)
##   otel          0.2.0   2025-08-29 [2] RSPM (R 4.6.0)
##   png           0.1-9   2026-03-15 [2] RSPM (R 4.6.0)
##   Rcpp          1.1.2   2026-07-05 [2] RSPM (R 4.6.0)
##   reticulate    1.47.0  2026-09-03 [2] RSPM (R 4.6.0)
##   rlang         1.3.0   2026-07-05 [2] RSPM (R 4.6.0)
##   rmarkdown     2.32    2026-09-01 [2] RSPM (R 4.6.0)
```

```
## sessioninfo 1.2.4 2026-06-04 [2] RSPM (R 4.6.0)
## xfun 0.60 2026-07-09 [2] RSPM (R 4.6.0)
## yaml 2.3.12 2025-12-10 [2] RSPM (R 4.6.0)
##
## [1] /tmp/RtmpopvETa/Rinst8e262e157d
## [2] /usr/local/lib/R/site-library
## [3] /usr/local/lib/R/library
##
## -----
```

3 Containerizing and Publishing against specific Bioconductor release versions

3.1 For a {BiocBook} package not currently in Bioconductor

One can build against older Bioconductor releases as follows:

- Commit current changes before switching branches (should be in `devel` branch)

```
gert::git_commit_all("Commit current changes")
```

- Create a new branch named `RELEASE_X_Y` and checkout

```
gert::git_branch_create("RELEASE_3_17")
```

- Push local `RELEASE_X_Y` to Github

```
gert::git_push()
```

3.2 For a {BiocBook} package accepted in Bioconductor > 6 months ago

Once your package is accepted in Bioconductor, your repository will have access to a new **remote** named `upstream`, pointing to `git@git.bioconductor.org`. When Bioconductor releases a new version, your package will change version on the `upstream` remote, in a dedicated release branch `RELEASE_X_Y`. All details are available [here](#).

To generate a Docker image and a version of the BiocBook website built on new Bioconductor releases, you can:

- Add the `Bioconductor` remote to your local repository (this might already be set up)

```
gert::git_remote_add(name = 'upstream', url = 'git@git.bioconductor.org:packages/<YOUR-REPOS
```

- Create a new local RELEASE_X_Y branch

```
gert::git_branch_create("RELEASE_3_15")
```

- Pull the upstream RELEASE_X_Y commits

```
gert::git_pull("RELEASE_3_15", remote = "upstream")
```

- Push the local RELEASE_X_Y branch to origin remote (your own Github repository)

```
gert::git_push("RELEASE_3_15", remote = "origin")
```

3.3 Session info

```
sessioninfo::session_info()
```

4 Executing python code

Important points

- BiocBooks can execute **python code**, not just R code;
- The code runs on **every render**, including when the *Bioconductor Build System* builds the book;
- The **python** packages the book needs are therefore installed **at build time**, by the book itself, from the conda environment declared in `inst/requirements.yml`;
- **Keep at least one R chunk on a python page**: it is what keeps quarto on the knitr engine.

4.1 Declaring and installing the python environment

`python` packages are listed, pinned, in `inst/requirements.yml` – a plain conda environment file, e.g.

```
name:
  BiocBook
channels:
  - conda-forge
  - bioconda
  - nodefaults
dependencies:
  - python=3.12
  - numpy=1.26
  - matplotlib
```

Note the presence of the `nodefaults` channel: the `defaults` channel is covered by the Anaconda Terms of Service, which require a paid licence for many organisations. `conda-forge` and `bioconda` are community channels that are free to use.

The first page that needs `python` installs and activates them:

```
library(reticulate)
BiocBook::setup_python()
## v Using the `python` already configured for this session: '/opt/R-cache/R/BiocBook/envs/1
```

`setup_python()` finds `inst/requirements.yml`, creates the `conda` environment declared there (if not installed yet) and activates it, addressing it by full path rather than by name. If the environment already exists, it is re-used rather than resolved again, and if `RETICULATE_PYTHON` is already set (as it is inside the book's `Docker` image) the whole step short-circuits.

4.2 Where conda itself comes from

Building a `conda` environment needs a `conda` implementation, and the machines that build this book do not necessarily have one. Bioconductor's current builders may, and the `r-universe` build image ships `quarto`, `python3` and `pip`, but no `conda` at all. So `BiocBook` brings its own.

`micromamba` is a single self-contained binary: no `conda` installation, no base environment, no `python`.

`BiocBook::micromamba()` resolves it in order:

1. `RETICULATE_CONDA`, if you have set it;
2. a `micromamba` already on the `PATH` (this book's `Docker` image installs one);
3. a copy previously downloaded into `BiocBook`'s cache;
4. failing all of those, it downloads a **pinned** release and checks it against a recorded `SHA256` before using it.

i Keep an R chunk on the page

`quarto` chooses its execution engine **per file**. A page with at least one R chunk uses `knitr`, and runs its `python` chunks through `reticulate`. A page whose only code is `python` uses `jupyter` instead. However, the Bioconductor builders do not provide `jupyter`, so a `jupyter` page cannot be rendered there at all.

To force `quarto` to use `knitr`, keep at least one R chunk on the page, even if it is empty.

4.3 A worked example

`reticulate` hosts a single, persistent `python` session that is shared with R, so objects can be passed in both directions.

Start from an R object:

```
counts <- matrix(
  c(12, 0, 45, 3, 7, 91, 0, 5, 33),
  nrow = 3,
  dimnames = list(c("geneA", "geneB", "geneC"), c("s1", "s2", "s3"))
)
counts
##           s1 s2 s3
## geneA  12  3  0
## geneB   0  7  5
## geneC  45 91 33
```

The R object is reachable from python through the `r` object:

```
import numpy as np
counts = np.array(r.counts)
print("shape:", counts.shape)
## shape: (3, 3)
print("library sizes:", counts.sum(axis = 0))
## library sizes: [ 57. 101.  38.]
```

State persists from one python chunk to the next, exactly as it would in a notebook:

```
cpm = counts / counts.sum(axis = 0) * 1e6
print("CPM values:")
## CPM values:
print(np.round(cpm, 1))
## [[210526.3  29703.      0. ]
##   [      0.  69306.9 131578.9]
##   [789473.7 900990.1 868421.1]]
```

Finally, the python object can be passed back to R:

```
cpm <- reticulate::py$cpm
cpm
##           [,1]      [,2]      [,3]
## [1,] 210526.3 29702.97      0.0
## [2,]      0.0 69306.93 131578.9
## [3,] 789473.7 900990.10 868421.1
```

4.4 Bioconda packages

Because the BiocBook environment is a conda environment, it can install any package from the bioconda channel, e.g. deeptools:

```
from importlib.metadata import version
v = version("deeptools")
```

Then read that in R:

```
reticulate::py$v
## [1] "3.5.5"
```

4.5 Session info

```
sessioninfo::session_info()
## - Session info -----
##   setting    value
##   version    R version 4.6.1 (2026-06-24)
##   os         Ubuntu 24.04.4 LTS
##   system     x86_64, linux-gnu
##   ui         X11
##   language   (EN)
##   collate    C
##   ctype      en_US.UTF-8
##   tz         Etc/UTC
##   date       2026-09-04
##   pandoc     3.10.2 @ /usr/bin/ (via rmarkdown)
##   quarto     1.9.38 @ /usr/local/bin/quarto
##
## - Packages -----
##   package      * version date (UTC) lib source
##   askpass       1.2.1   2024-10-04 [2] RSPM (R 4.6.0)
##   BiocBook      1.11.1  2026-09-04 [2] Github (js2264/BiocBook@562ccc6)
##   BiocGenerics  0.59.12 2026-08-11 [2] Bioconductor 3.24 (R 4.6.1)
##   cli           3.6.6   2026-04-09 [2] RSPM (R 4.6.0)
##   credentials   2.0.3   2025-09-12 [2] RSPM (R 4.6.0)
##   digest        0.6.39  2025-11-19 [2] RSPM (R 4.6.0)
##   dplyr         1.2.1   2026-04-03 [2] RSPM (R 4.6.0)
```


##	<i>evaluate</i>	1.0.5	2025-08-27	[2]	RSPM	(R 4.6.0)
##	<i>fastmap</i>	1.2.0	2024-05-15	[2]	RSPM	(R 4.6.0)
##	<i>fs</i>	2.1.0	2026-04-18	[2]	RSPM	(R 4.6.0)
##	<i>generics</i>	0.1.4	2025-05-09	[2]	RSPM	(R 4.6.0)
##	<i>gert</i>	2.4.1	2026-08-19	[2]	RSPM	(R 4.6.0)
##	<i>gh</i>	1.6.1	2026-07-20	[2]	RSPM	(R 4.6.0)
##	<i>gitcreds</i>	0.1.2	2022-09-08	[2]	RSPM	(R 4.6.0)
##	<i>glue</i>	1.8.1	2026-04-17	[2]	RSPM	(R 4.6.0)
##	<i>htmltools</i>	0.5.9	2025-12-04	[2]	RSPM	(R 4.6.0)
##	<i>httr</i>	1.4.9	2026-09-01	[2]	RSPM	(R 4.6.0)
##	<i>jsonlite</i>	2.0.0	2025-03-27	[2]	RSPM	(R 4.6.0)
##	<i>knitr</i>	1.51	2025-12-20	[2]	RSPM	(R 4.6.0)
##	<i>lattice</i>	0.23-1	2026-08-12	[2]	RSPM	(R 4.6.0)
##	<i>lifecycle</i>	1.0.5	2026-01-08	[2]	RSPM	(R 4.6.0)
##	<i>magrittr</i>	2.0.5	2026-04-04	[2]	RSPM	(R 4.6.0)
##	<i>Matrix</i>	1.7-6	2026-07-25	[2]	RSPM	(R 4.6.0)
##	<i>openssl</i>	2.4.2	2026-06-09	[2]	RSPM	(R 4.6.0)
##	<i>otel</i>	0.2.0	2025-08-29	[2]	RSPM	(R 4.6.0)
##	<i>pak</i>	0.11.1	2026-07-22	[2]	RSPM	(R 4.6.0)
##	<i>pillar</i>	1.11.1	2025-09-17	[2]	RSPM	(R 4.6.0)
##	<i>pkgconfig</i>	2.0.3	2019-09-22	[2]	RSPM	(R 4.6.0)
##	<i>png</i>	0.1-9	2026-03-15	[2]	RSPM	(R 4.6.0)
##	<i>purrr</i>	1.2.2	2026-04-10	[2]	RSPM	(R 4.6.0)
##	<i>R6</i>	2.6.1	2025-02-15	[2]	RSPM	(R 4.6.0)
##	<i>Rcpp</i>	1.1.2	2026-07-05	[2]	RSPM	(R 4.6.0)
##	<i>renv</i>	1.2.4	2026-08-03	[2]	RSPM	(R 4.6.0)
##	<i>reticulate</i>	* 1.47.0	2026-09-03	[2]	RSPM	(R 4.6.0)
##	<i>rlang</i>	1.3.0	2026-07-05	[2]	RSPM	(R 4.6.0)
##	<i>rmarkdown</i>	2.32	2026-09-01	[2]	RSPM	(R 4.6.0)
##	<i>rprojroot</i>	2.1.1	2025-08-26	[2]	RSPM	(R 4.6.0)
##	<i>sessioninfo</i>	1.2.4	2026-06-04	[2]	RSPM	(R 4.6.0)
##	<i>stringi</i>	1.8.9	2026-08-04	[2]	RSPM	(R 4.6.0)
##	<i>stringr</i>	1.6.0	2025-11-04	[2]	RSPM	(R 4.6.0)
##	<i>sys</i>	3.4.3	2024-10-04	[2]	RSPM	(R 4.6.0)
##	<i>tibble</i>	3.3.1	2026-01-11	[2]	RSPM	(R 4.6.0)
##	<i>tidyselect</i>	1.2.1	2024-03-11	[2]	RSPM	(R 4.6.0)
##	<i>usethis</i>	3.2.1	2025-09-06	[2]	RSPM	(R 4.6.0)
##	<i>vctrs</i>	0.7.3	2026-04-11	[2]	RSPM	(R 4.6.0)
##	<i>withr</i>	3.0.3	2026-06-19	[2]	RSPM	(R 4.6.0)
##	<i>xfun</i>	0.60	2026-07-09	[2]	RSPM	(R 4.6.0)
##	<i>yaml</i>	2.3.12	2025-12-10	[2]	RSPM	(R 4.6.0)
##						

```

## [1] /tmp/RtmpopvETa/Rinst8e262e157d
## [2] /usr/local/lib/R/site-library
## [3] /usr/local/lib/R/library
## * -- Packages attached to the search path.
##
## - Python configuration -----
## python: /opt/R-cache/R/BiocBook/evals/BiocBook/bin/python
## libpython: /opt/R-cache/R/BiocBook/evals/BiocBook/lib/libpython3.12.so
## pythonhome: /opt/R-cache/R/BiocBook/evals/BiocBook:/opt/R-cache/R/BiocBook/evals/BiocBook
## version: 3.12.14 (main, Sep 2 2026, 23:27:36) [GCC 15.3.0]
## numpy: /opt/R-cache/R/BiocBook/evals/BiocBook/lib/python3.12/site-packages/numpy
## numpy_version: 1.26.4
##
## NOTE: Python version was forced by RETICULATE_PYTHON
##
## -----

```

5 {BiocBook}-based books vs. {rebook}-based books

{rebook} is another book rendering package currently used by Bioconductor to build book packages such as `OSCA.*` and `SingleRBook`.

5.1 Differences with {rebook}-based books

OSCA books provided by Bioconductor are rendered upon building by the *Bioconductor Build System* (BBS). They rely on {rebook} to orchestrate the rendering. Briefly, OSCA books have their book pages in `/inst/book/`, while the `vignettes/` folder contains (1) a `Makefile` and (2) a dummy `stub.Rmd` vignette (required to trigger vignette rendering and thus `make`).

When the BBS triggers `OSCA.intro` package building, upon vignette rendering, the `Makefile` triggers the following commands:

Listing 5.1 R

```
work.dir <- rebook::bookCache('OSCA.intro')
handle <- rebook::preCompileBook('../inst/book', work.dir=work.dir, desc='../DESCRIPTION')
old.dir <- setwd(work.dir)
bookdown::render_book('index.Rmd')
setwd(old.dir)
rebook::postCompileBook(work.dir=work.dir, final.dir='../inst/doc/book', handle=handle)
```

The resulting book, pre-compiled by {rebook} and assembled by {bookdown}, is eventually served by Bioconductor from the `/inst/doc/book/` folder.

{BiocBook}-based packages follow a strategy similar to that of OSCA books: they provide a `Makefile` in the `vignettes/` folder to trigger book rendering when building the package. However, the executed command does not rely on {rebook} and {bookdown}, but on a `render` command from the `quarto` software.

The resulting book, fully compiled by native `quarto`, is also located in `/inst/doc/book/` once the package is built (and in `doc/book` in the library directory once installed).

Listing 5.2 shell

```
quarto render ../inst/  
mv ../inst/docs ../inst/doc/book
```

⚠ This implies that `quarto` (≥ 1.3) has to be installed in the system building the package!

5.2 BiocBook features missing from rebook

- A BiocBook can be readily initiated using `BiocBook::init()`;
- It relies on modern `.qmd` files supported by Quarto;
- It can work as a standalone Github-hosted package, without necessarily having to be submitted to/built by Bioconductor. The book should be rendered exactly the same way through Github or by the BBS;
- It supports versioning of the online book served by `gh-pages` **through the author Github account**;
- It distributes versioned Dockerfiles **through the author Github account**;
- BiocBook-based packages can actually provide fully-fledged functions in R/, manual pages and vignettes. They can be installed exactly the same way than other software packages.

5.3 rebook features missing from BiocBook

- Smart reuse of objects generated in one book in another book;

💡 Tip

This can still be achieved **within** a book by saving a data object as an `.rds` file and loading it in a subsequent chapter.

```
isthisworking <- readRDS('isthisworking.rds')  
isthisworking  
## [1] "yes"
```

- Native support of cross-references across books.

5.4 Session info

```
sessioninfo::session_info()
## - Session info -----
##   setting    value
##   version    R version 4.6.1 (2026-06-24)
##   os         Ubuntu 24.04.4 LTS
##   system      x86_64, linux-gnu
##   ui          X11
##   language    (EN)
##   collate     C
##   ctype       en_US.UTF-8
##   tz          Etc/UTC
##   date        2026-09-04
##   pandoc      3.10.2 @ /usr/bin/ (via rmarkdown)
##   quarto      1.9.38 @ /usr/local/bin/quarto
##
## - Packages -----
##   package      * version date (UTC) lib source
##   cli           3.6.6   2026-04-09 [2] RSPM (R 4.6.0)
##   digest        0.6.39  2025-11-19 [2] RSPM (R 4.6.0)
##   evaluate      1.0.5   2025-08-27 [2] RSPM (R 4.6.0)
##   fastmap       1.2.0   2024-05-15 [2] RSPM (R 4.6.0)
##   htmltools     0.5.9   2025-12-04 [2] RSPM (R 4.6.0)
##   jsonlite      2.0.0   2025-03-27 [2] RSPM (R 4.6.0)
##   knitr         1.51    2025-12-20 [2] RSPM (R 4.6.0)
##   otel          0.2.0   2025-08-29 [2] RSPM (R 4.6.0)
##   rlang         1.3.0   2026-07-05 [2] RSPM (R 4.6.0)
##   rmarkdown     2.32    2026-09-01 [2] RSPM (R 4.6.0)
##   sessioninfo   1.2.4   2026-06-04 [2] RSPM (R 4.6.0)
##   xfun          0.60    2026-07-09 [2] RSPM (R 4.6.0)
##   yaml          2.3.12  2025-12-10 [2] RSPM (R 4.6.0)
##
##   [1] /tmp/RtmpopvETa/Rinst8e262e157d
##   [2] /usr/local/lib/R/site-library
##   [3] /usr/local/lib/R/library
##
## -----
```